

End-to-End MLOps Pipeline — Emotion Classification

MLOps Assignment 3 · Group 9 · PGD AI · IIT Jodhpur

1. Team & contributions

Roll No.	Name	Contribution
G25ait2083	Rashmi Kumari	Repo admin, branch protection, CI/CD triggering, Hugging Face model registry, report (Task 1, 5, 7)
G25ait2046	Kanwaldeep Singh	Data prep & normalisation, model loading, inference script + Dockerfile, report (Task 2, 3, 6)
G25AIT2031	Disha Singhania	Kaggle training runs co-authoring & execution, W&B logging & dashboard setup, release merging, GitHub Actions workflow execution, report (Task 4, 8, 7)
G25AIT2005	Abhishek Virmani	Not Present

2. Live links

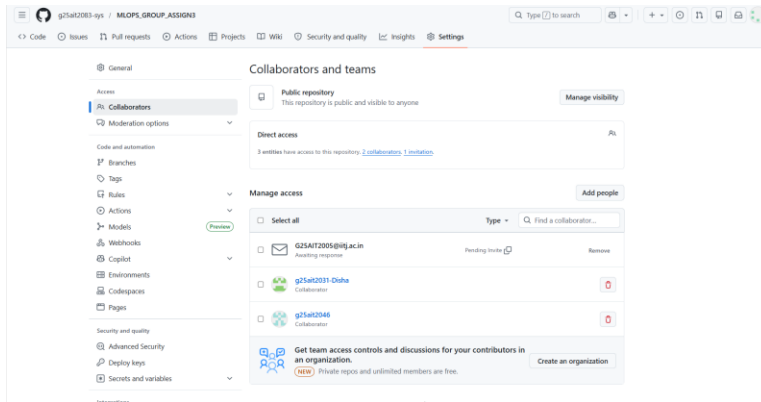
Component	URL
GitHub repository	https://github.com/g25ait2083-sys/MLOPS_GROUP_ASSIGN3
Kaggle notebook — V1	https://www.kaggle.com/code/g25ait2046/notebook72b692c7fd
Kaggle notebook — V2	https://www.kaggle.com/code/g25ait2046/notebook72b692c7fd
Hugging Face model	https://huggingface.co/g25ait2046/distilbert-emotion-mlops-a3
Docker image	https://hub.docker.com/r/rashmi2083/mlops-emotion-inference
W&B project dashboard	https://wandb.ai/g25ait2083-iit/mlops-assignment3

3. Git repository setup (Task 1)

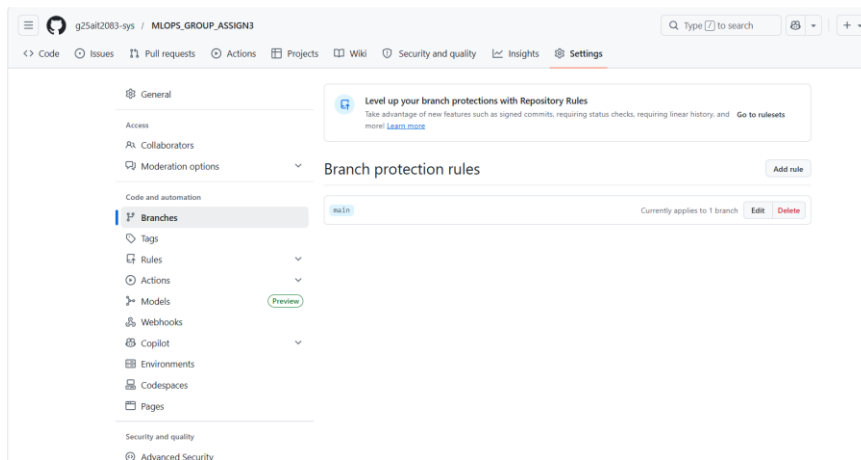
- Public repo with README.md, .gitignore, LICENSE.
- Branch strategy: main is the stable branch; develop is the integration branch where all code is pushed before review.
- Branch protection on main: at least 1 pull-request review required before merging.
- Owner(g25ait2083-sys) is Admin; all teammates added as Collaborators with Write access.

- **Branch Integration & Release Process:** To maintain repository integrity, all feature development took place on dedicated branches. Code was integrated and verified on the develop branch. Final release deployment to main was controlled via a GitHub Pull Request, requiring a mandatory review approval and passing CI status checks before merging, ensuring no unverified code reached production.

Screenshot 1 — Collaborators & roles:



Screenshot 2 — Branch protection on main:



4. Data cleaning & normalisation (Task 2)

Dataset: dair-ai/emotion (split config) — 20,000 short English texts labelled with one of six emotions. Splits: 16,000 train / 2,000 validation / 2,000 test. Labels are read directly from the dataset's ClassLabel schema: 0=sadness, 1=joy, 2=love, 3=anger, 4=fear, 5=surprise.

Inspection — class distribution (counts and % per split):

Emotion	Train (n / %)	Validation (n / %)	Test (n / %)
sadness	4666 / 29.2%	550 / 27.5%	581 / 29.1%
joy	5362 / 33.5%	704 / 35.2%	695 / 34.8%
love	1304 / 8.2%	178 / 8.9%	159 / 8.0%
anger	2159 / 13.5%	275 / 13.8%	275 / 13.8%
fear	1937 / 12.1%	212 / 10.6%	224 / 11.2%
surprise	572 / 3.6%	81 / 4.0%	66 / 3.3%
Total	16000	2000	2000

The corpus is clearly class-imbalanced: joy (33.5%) and sadness (29.2%) dominate, while surprise is the rarest at just 3.6% of the training set. The split proportions are consistent across train/validation/test, so the imbalance is inherent to the data rather than a bad split. There were no missing labels.

Cleaning decisions (and why):

- Lowercasing — we fine-tune distilbert-base-uncased, which lowercases internally; normalising on disk keeps stored data faithful to model input.
- Strip URLs and @mentions — they carry no emotional signal and add noise.
- Collapse repeated whitespace — uniform spacing, no semantic change.
- Drop empty rows after cleaning — none occurred here, but the guard is in place.
- Drop exact-duplicate texts — prevents the same sample leaking across splits, which would inflate metrics.
- Punctuation kept by default — '!' and '?' are genuine emotion cues; an opt-in --strip-punctuation flag is available if needed.

Cleaning summary (actual output of src/data_prep.py):

Split	Rows in	Rows out	Dropped: missing	Dropped: duplicates
train	16000	15969	0	31
validation	2000	1998	0	2
test	2000	2000	0	0

Cleaning removed only 33 exact-duplicate rows in total and no rows became empty, confirming the corpus was already fairly clean. The id2label mapping is saved to data/id2label.json (the only data artefact committed to git).

5. Model selection rationale (Task 3)

We chose distilbert-base-uncased. According to its Hugging Face model card, DistilBERT is a distilled version of BERT-base that retains roughly 97% of BERT's language-understanding performance while being ~40% smaller and ~60% faster. At ~66M parameters (~250 MB) it fine-

tunes comfortably within Kaggle's free T4 GPU quota, and short emotion tweets fit easily inside a 128-token window, so the speed/size trade-off costs us almost nothing in accuracy. It is uncased — appropriate for casual social-media text where capitalisation is inconsistent — and the model card documents that it was pretrained on the same BookCorpus + English Wikipedia data as BERT via knowledge distillation. The AutoModelForSequenceClassification head is initialised with our six labels and the id2label/label2id maps, so the deployed model emits human-readable class names rather than LABEL_0...LABEL_5.

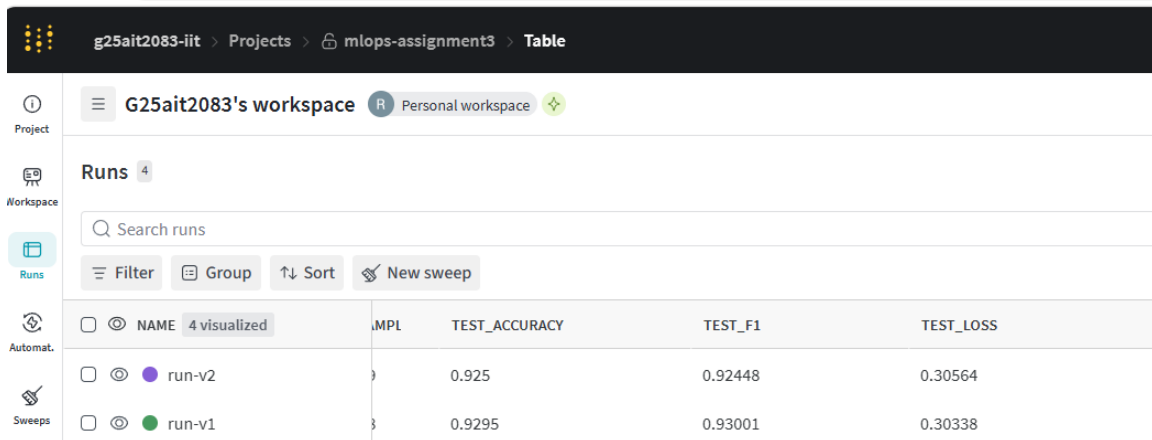
6. Experiment comparison (Task 4)

Hyperparameter	Version 1	Version 2
Learning rate	3e-5	5e-5
Train batch size	16	32
Epochs	3	4
Weight decay	0.0	0.01

Metric (held-out test)	Version 1	Version 2
Accuracy	92.95%	92.50%
Weighted F1	0.9300	0.9245
Validation loss	0.3034	0.3056

Winner: Version 1 performed better with higher accuracy (92.95% vs 92.50%) and F1 (0.9300 vs 0.9245). The higher learning rate in V2 (5e-5) likely caused slight overfitting despite the weight decay regularisation, while V1's conservative learning rate (3e-5) converged more stably on the imbalanced emotion classes.

Screenshot 3 — W&B dashboard showing both runs:



g25ait2083-ijt > Projects > mlops-assignment3 > Table				
G25ait2083's workspace Personal workspace				
Runs 4				
Search runs				
Filter Group Sort New sweep				
☐	NAME 4 visualized	IMPL	TEST_ACCURACY	TEST_F1
☐	run-v2		0.925	0.92448
☐	run-v1		0.9295	0.93001
				TEST_LOSS
				0.30564
				0.30338

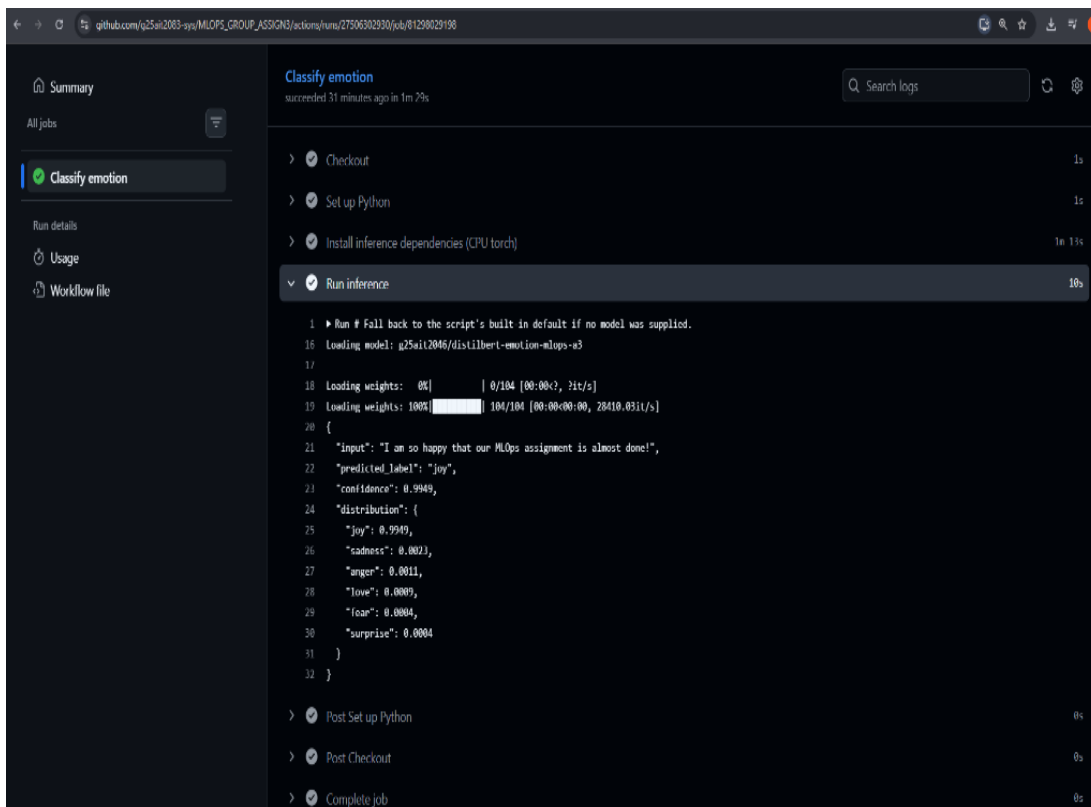
7. Docker design (Task 6)

- Base: python:3.11-slim — small and matches the CI Python version.
- CPU-only PyTorch installed from download.pytorch.org/whl/cpu so the image is a few hundred MB instead of multi-GB CUDA.
- ARG HF_MODEL_NAME with a sensible public default, overridable at build.
- Installs only requirements.txt (inference deps) — no training stack.
- Runs as a non-root user; INPUT_TEXT supplied at run time.

Build, test and push commands:

```
docker build -t rashmi2083/mlops-emotion-inference:latest .
docker run --rm -e HF_TOKEN=<token> -e INPUT_TEXT="I am so happy!" rashmi2083/mlops-emotion-inference:latest
docker push rashmi2083/mlops-emotion-inference:latest
```

Screenshot 4 — successful GitHub Actions inference run (badge or log):



8. Challenges & learnings

- WSL 2 not installed — Docker Desktop failed to start on Windows 11 until WSL was installed via `wsl --install`.
- W&B version conflict — `wandb==0.17.0` was incompatible with Kaggle's Python 3.12; fixed by removing the version pin.
- NumPy 2.x conflict — GitHub Actions runner had `numpy>=2` which broke torch; fixed by pinning `numpy<2` in `requirements.txt`.
- File structure errors — files accidentally created inside `src/` instead of repo root; fixed by deleting and recreating at correct paths.
- Class imbalance — surprise is only 3.6% of training data making it the hardest class to predict correctly.

What we would do differently:

- Add class weights or oversampling for the minority surprise class to improve per-class F1.
- Change only one hyperparameter at a time between versions for cleaner attribution of performance differences.

- Use a W&B Sweep for systematic hyperparameter search instead of manual versions.

9. Conclusion

Our team successfully built and verified an end-to-end production-grade MLOps pipeline. By fine-tuning DistilBERT on the emotion dataset, containerizing the model in Docker, and automating testing via GitHub Actions, we established a reproducible workflow where all performance runs are transparently tracked and compared on Weights & Biases.